Programming Language Design

Module 6: More functional operators

Slides © Grant Williams, CC BY-SA 4.0.

This is an open educational resource.
Feel free to submit fixes, improvements, and new material here.

Last time

We learned about lexing. [what is it?]

We learned about functional operators. [what are they?]

This time

Way more functional operators! (I know you think they're awesome)

Tail recursion (an optimizable way to write recursive functions)

Tail recursion

Is recursion really good enough?

We know about for-loops. We also know they're pretty fast.

Function calls have overhead.

So doesn't using recursion everywhere slow us down?

Well, most of the time there really is some overhead to it. But not *always*...

Tail recursion (2)

There's a special time in which recursion doesn't necessarily require much more overhead than a loop does.

It's when the last thing we do (the "return") is the recursive call.

We refer to such a function call as a "tail call".

For example, this is *not* a tail call:

```
sum (x : xs) = x + sum xs
```

Because, after we call `sum xs` recursively, we then add x to the result.

However, what if we wrote it like this...

Tail recursion (3)

```
sum acc [] = acc
sum acc (x : xs) = sum (acc + x) xs
```

Here, we're still computing the sum. `sum 0 [1,2,3]` still returns `6`. However, the recursive call of sum is the last (and only) thing the function does. Adding `x` happens before.

Here, we call `acc` an *accumulator* (hence its name). We add each `x` value to the accumulator, and when we're out of elements, we return it.

Tail recursion (4)

Why is this faster?

Because the function's stack frame can be re-used.

Consider what would happen in the classic C calling convention:

```
int result = sum(0, p, n);
```

In old-school 32-bit `ccall`, we first push `n`, then `p`, then `0`.

Inside the function, we do the same thing for the recursive call.

And again...

So every time there's a recursive call, we push the function's data.

When it's done, we free the data by moving the stack pointer back up (adding to it).

Tail recursion (5)

There are two issues:

1. Pushing/moving the stack pointer takes a little time
2. We don't actually free that memory until the very end. The stack stays allocated.

Are these things bad? Well, number 1 is a small slowdown, but number 2 is a bigger problem. In C, there is a limited amount of stack memory.

In Haskell, there's quite a lot more stack space by default, but you can still have stack overruns if you're not careful.

Given the importance of recursion in functional programming, we like to use recursion heavily, and we don't want to run out of stack space.

Tail recursion (6)

So, if the *last thing* a function does is to return a recursive call, we can just re-use the same stack space:

```
int sum(int acc, int* vals, int n) {
    if (n <= 0) return acc;

    return sum(acc + *vals, vals + 1, n - 1);
}
```

Here, for the recursive call to `sum`, we can overwrite `acc` with `acc + *vals`, we can overwrite `vals` with `vals + 1` (which makes it point to the next value), and we can overwrite `n` with `n - 1`.

Tail recursion (7)

This function is *not* tail recursive, because the addition happens after the recursive call:

```
int sum(int* vals, int n) {
    if (n <= 0) return 0;
    return sum(vals + 1, n - 1)
}
```

So we have to keep pushing the pointer and `n` on the stack until finally, we reach `n == 0`, and we can return from each call.

Tail recursion (8)

In haskell, not tail recursive:

```
sumNoTail [] = 0
sumNoTail (x : xs) = x + sumNoTail xs
```

Tail recursive:

```
sumTail acc [] = acc
sumTail acc (x : xs) = sumTail (acc + x) xs
```

Is it worth it? Well, if we expect the list to be long, *maybe* it's worth optimizing. Sometimes, though, it's vastly faster to use tail recursion. We'll see an example soon.

But, even if you use tail recursion in Haskell, you can still crash from stack overflow. That's because of lazy evaluation.

Lazy eval and tail recursion

What happens in Haskell when we do this?

`y = sumTail 0 [1,2,3]`

Normally: nothing, because of lazy evaluation. However, suppose we actually use `y` later: `main = print y`. Now we need to compute `y` so we can print it.

We apply the definition of `sumTail`:

```
sumTail 0 [1,2,3] ==
sumTail (0 + 1) [2,3] ==
sumTail (0 + 1 + 2) [3] ==
sumTail (0 + 1 + 2 + 3) [] ==
    (0 + 1 + 2 + 3) -- <- actual answer
```

We don't actually combine `0 + 1 + 2 + 3` into one value until printing.

Lazy eval and tail recursion (2)

Why do we keep collecting values? Why does the accumulator end up being `0 + 1 + 2 + 3` instead of `6`? Because of lazy evaluation.

When we compute something like `x + 1`, Haskell doesn't immediately reduce that into a single value. Instead, it creates a *thunk* which is like an eager lambda function that, when run, will eventually return a value.

So here, the thunk would be a function that calls the thunk for `x` and then adds `1`.

This means that we still end up using extra space until the function returns, even when we're using tail recursion.

This is the real cost of lazy evaluation: it makes memory usage unpredictable. This is one reason why Haskell wouldn't be my first choice as a system or gamedev language.

Can we fix that?

We actually can just tell Haskell not to lazy-evaluate stuff using the `$!` operator.

This operator is almost identical to the `$` operator, but it forces its right argument to be eagerly evaluated by one stage, so you basically do one addition.

For example:

```
sumTail acc [] = acc
sumTail acc (x : xs) = (sumTail $! (acc + x)) xs
```

So now, because of the `$!`, we actually reduce the `acc + x` into a single value each time we recursive call, preventing the thunk from growing.

Foldl?

So, what about `foldl` and `foldr`? Remember when we defined those?

I don't, because we only defined `fold`, so here it is:

```
foldl'' :: (b -> a -> b) -> b -> [a] -> b
foldl'' _ init [] = init
foldl'' f acc (x : xs) = foldl'' f (acc `f` x) xs

foldr'' :: (a -> b -> b) -> b -> [a] -> b
foldr'' _ init [] = init
foldr'' f acc (x : xs) = f x (foldr'' f acc xs)
```

Notice that `foldl` is tail-recursive and `foldr` is not.

So does that mean we should use `foldl` when we can? Because it's tail recursive? Well, not really... I mean, first, we should notice that it's lazy, not eager!

Foldl (2)

There's actually another version of `foldl` that forces strict evaluation.

You have to `import Data.List` to use it...

It's called...[drumroll]

`foldl'`

It's the same as `foldl`, but it's strict. So `sum = foldl' (+) 0` won't stack overflow.

There's also `foldr'`, but it seems less useful to me, because it's still not tail recursive, so the stack keeps growing.

Should I use it?

So if `foldl'` is tail-recursive and eager, that must be the fastest one, right?

It makes sense, but we should test it.

I'm going to make a simple microbenchmark, but first, **big warning**: microbenchmarks are highly specific. They show you how a language performs on a very specific task on a specific platform at a specific point in time and phase of the moon.

I'm going to show you what happens when we use `foldl'` instead of `foldl` specifically to sum a giant list of `Int` (not `Integer`) *and* the list has already been fully constructed (not lazily) *and* it's running on Windows *and* probably a bunch of other stuff I haven't controlled for.

A microbenchmark

```
main :: IO ()
main = do
    let count = 100000000
    let xs = [1..count] :: [Int]
    deepseq xs (return ()) -- force construct list first: I'll explain this

    start <- getCurrentTime
    print $ foldl (+) 0 xs
    end <- getCurrentTime
    putStrLn $ "foldl time: " ++ show (diffUTCTime end start)

    start <- getCurrentTime
    print $ foldl' (+) 0 xs
    end <- getCurrentTime
    putStrLn $ "foldl' time: " ++ show (diffUTCTime end start)
```

Note: if you want to do this, import `Data.List`, `Data.Time`, and `Control.DeepSeq`

Results

On my specific computer I get this:

```
5000000050000000
foldl time: 15.7061889s
5000000050000000
foldl' time: 0.5516658s
```

So it actually made a huge difference.

But note: *big* list. I've seen another person online run this test in different circumstances online and they said that it actually slowed it down.

Also, in other news, `foldr` is actually faster for me than `foldl`. Probably because it doesn't end up needing to build thunks for each addition.

Okay, now let me answer your other question...

What is `deepseq`

`deepseq` is related to `$!`.

In fact, there is a function called `seq` that is closely related to `$!`.

`seq a b` means "slightly simplify `a`, then return `b`

This technically means it has a side effect. It is one of the few haskell "functions" with side effects. If they are observable, you probably don't want to use it.

We can define `$!` in terms of `seq`: `f $! x = f (seq x x)`

What is "slightly simplify"? In the case of a list, it will remove the outer thunk and expose the *cons*: `seq (1 : 2 : []) (1 : 2 : [])` will create a single list node with a value of `1`, whose next pointer is a thunk that will construct `2 : []`

What is `deepseq`? (2)

That's all `seq` does. Calling it again won't help. It just consumes the outer thunk.

For arithmetic operations, it actually does simplify the whole thing, but those are a special case.

Since seq isn't that helpful, if we want to thoroughly construct the entire list, we use `deepseq`.

This consumes the thunk and replaces it with a fully initialized list, so that we can compare `foldl` and `foldl'` directly without also including the time it takes to allocate list nodes.

`deepseq x (return ())` means "first compute `x`, and then do nothing". `return` in Haskell does not mean what it means in C. It's a constructor for monads. In this case, an IO object that does nothing and returns `()`. We'll talk about this later.

$!!

In the same way that `$!` does `seq` before calling a function, `$!!` does `deepseq`.

We don't need to do these things often, but it can occassionally be important, especially when doing performance optimization.

Questions?

Knowledge check 1

1. Define a function `reverse'`, which should take a list an reverse it. Include its type. Do not use tail recursion.
2. What is the big-O runtime of this function?
3. Define a tail-recursive version. Include its type.
4. What is the big-O runtime of this function? This is the one I warned you about earlier when I said tail recursion would have a huge impact on performance.
5. Now use one of the folds to define `reverse`.
6. Suppose we had a giant list we wanted to reverse. How could we force evaluation to reverse it before we needed to use it later.

KC 1 answers (1)

1.

```
reverse' :: [a] -> [a]
reverse' [] = []
reverse' (x : xs) = reverse' xs ++ [x]
```

2. It's quadratic! Because `++` is linear and we're doing it `n` times, where `n` is the length of the list. It's much slower than you'd think.
3.

```
reverse'' :: [a] -> [a] -> [a]
reverse'' acc [] = acc
reverse'' acc (x : xs) = reverse'' (x : acc) xs
```

KC 1 answers (2)

4. This one is actually linear. We're prepending each value in the list to the front of the accumulator, which is fast (constant time). We do this for each value in the list.
5.

```
reverse''' :: [a] -> [a]
reverse''' = foldl (flip (:)) []
```

More explanation about `flip` on the next slide.

`flip`

`flip` is a function that takes a function and flips its arguments. It's defined like this: `flip f = \y x -> f x y`. So instead of `:` taking a value and a list and prepending the left argument to the right, `flip (:)` returns a function that takes a list and then a value to prepend.

We run this on each value of the list. It's like using a stack to reverse a list. We push each value from left to right onto the front of the return list.

[can you do it without flip given the definition above?]

KC 1 answers (3)

6. `let reversed = deepseq (reverse l) (reverse l) in ...`

Here, `deepseq` makes it so that the list is fully allocated and not just a bunch of thunks that perform the `cons` operation.

It's really not required to do this. Remember that premature optimization is the root of all evil.

Questions?

More operators: zip

I know what you're thinking.

"We love functional programming, but there aren't enough higher order functions. Please teach us another functional operator!"

Okay, let's learn about `zip`. This is a surprisingly useful operator that doesn't show up very often in imperative languages (although Python has it).

`zip`

Sometimes we have two lists that we want to "pair up" somehow.

For example, maybe there are a lists of names, and their employee ids and we want to print them out together.

First, let's see how we would do this in C...

Pairing up lists in C

The most common way I see imperative programmers solve this problem is by using the same index in both lists.

```
char* names[] = { "Alice", "Bob", "Camille", "Dan", ... };
char* eids[] = { "1234", "5678", "9123", "4567", ... }

void print_names_and_eids(char** names, char** ids, size_t n) {
    for (size_t i = 0; i < n; i++) {
        printf("%s: %s\n", names[i], ids[i]);
    }
}
```

Here, `i` is an index (a `size_t`, which is usually an `unsigned long`).

We iterate through both lists with the same index, so we "pair up" names and eids which are in the same order.

Pairing up lists in Haskell

There is no reason we can't do this in Haskell, too:

```
printNamesAndEids :: [String] -> [String] -> IO ()
printNamesAndEids _ [] = return ()
printNamesAndEids [] _ = return ()
printNamesAndEids (name : names) (eid : eids) = do
    putStrLn $ name ++ ": " ++ eid
    printNamesAndEids names eids
```

(Note: I don't expect you to understand what `return ()` does yet, or what an `IO ()` really is. I'm just showing you that Haskell can do the same thing as C.)

However, as you might have started noticing, even though recursion is important to functional programming, we usually end up finding elegant ways of solving problems that handle the recursion for us.

Using `zip` to pair up lists

That is what we will do here. We will use `zip`.

`zip` is a function that takes a pair of lists, and returns a list of pairs.

That is, `zip ["Alice", "Bob", "Camille"] ["123", "456", "789"] == [("Alice", "123"), ("Bob", "456"), ("Camille", "789")]`

Let's do a bit of clean up.

Using `zip` to pair up lists

```haskell
import Control.Monad -- we will learn more about monads later. Just a taste!
names = ["Alice", "Bob", "Camille"]
ids = ["123", "456", "789"]

namesAndEids :: [String]
namesAndEids = map (\(n, e) -> n ++ ": " ++ e) $ zip names eids

printNamesAndEids' :: IO ()
printNamesAndEids' = mapM_ putStrLn $ namesAndEids names ids
```

Don't worry to much about the `mapM_` function. It's kind of like Haskell's version of "foreach" in that it applies a function to a list.

The important thing is that `map ... zip` above...

Using `zip` (2)

Here it is again

```
namesAndEids :: [String]
namesAndEids = map (\(n, e) -> n ++ ": " ++ e) $ zip names eids
```

`zip names eidss` is returning a list of pairs.

Then, `map (\(n, e) -> n ++ ": " ++ e) ...` is applying that lambda function to each pair. It is taking the name and eid and pasting them together with `": "`.

The result is just a list of strings like "Alice: 123", "Bob: 456", "Camille: 789"

`zip` and `map` together

Using map with zip is so common, there's a special combination operator: `zipWith`:

```
namesAndEids' = zipWith (\n e -> n ++ ": " ++ e) names ids
```

It also automatically uncurries the function, so we can write `\n e` instead of `\(n, e)`.

[what is currying and uncurrying again?]

Knowledge check 2

1. Use zip to create a list of `(n, n^2)` for each natural number `n`. This should be an infinite list.
2. Print the first 10 elements of that infinite list.
3. Now construct a new infinite list that is the sum of each element of the first list. So `(0 + 0^2), (1 + 1^2), (2 + 2^2), (3 + 3^2), ...`
4. Print the first 10 elements of this infinite list.
5. Now, use `zipWith` to construct the same list as 3 without needing `uncurry`.

KC 2 answers

1.

```
nAndN2 :: [(Integer, Integer)]
nAndN2 = zip [0..] $ map (^2) [0..]
```

2. `print $ take 10 nAndN2`
3. `sumNAndN2 = map (uncurry (+)) nAndN2`
4. `print $ take 10 sumNAndN2`
5. `sumNAndN2' = zipWith (+) [0..] $ map (^2) [0..]`

Questions?

Making `zip`

Can we define `zip` ourselves?

First, [what should its type be?]

Making `zip`

```
zip :: [a] -> [b] -> [(a, b)]
```

"Give me a list of `a`s and a list of `b`s and I will give you a list of `(a, b)` pairs.

Now, [code it up]. First, let's use recursion...

Making `zip`

```
zip :: [a] -> [b] -> [(a, b)]
zip _ [] = []
zip [] _ = []
zip (x : xs) (y : ys) = (x, y) : (zip xs ys)
```

If either list is empty, we're done.

Otherwise, we pair up the head of both remaining lists, and append it to the end of the result.

This is the most straightforward way to do it. It's hard to use one of the `fold`s to make `zip`, because it takes 2 list arguments instead of 1.

It's technically possible, but it involves some goofy coding, like having the accumulator be a tuple of the 2nd list and an empty list of pairs and such. It's easier recursively.

Zip on infinite lists

Remember when we talked about the Fibonacci series?

Here was our definition of a Fibonacci function:

```
fibo :: [Integer] -> [Integer] -> [Integer]
fibo 0 = 0
fibo 1 = 1
fibo n = fibo (n - 1) + fibo (n - 2)
```

This works, but it's very slow. It re-calculates Fibonacci terms. It ends up taking exponential time (we'll cover the details of that in CS 450).

We can make it faster by using two accumulators to store the previous two values in the series [how?]

`fibo` with accumulator

Like this:

```
fibo n = fibo' n 0 1
    where
    fibo' 0 _ b = b
    fibo' n a b = fibo' (n - 1) b (a + b)
```

This is much faster. We are basically adding the previous 2 numbers in the list `n` times.

But it turns out there's a way to do this with raw lists. We don't need a function. We can just store the Fibonacci series as an infinite list using `zip` and lazy evaluation.

Any ideas how? [this one is a brain-bender, but instructive]

Here's how

```haskell
fibo :: [Integer]
fibo = [0, 1] ++ zipWith (+) fibo (drop 1 fibo)

main = print $ take 100 fibo
```

This actually works. It generates an infinite list of Fibonacci numbers. It's also reasonably fast (it doesn't require exponential time).

But why? Well, `fibo` is the concatenation of two lists: `[0, 1]` and that `zipWith` term.

`[0, 1]` just tells us the first two values.

But then the real clever trick happens...

Lazy `fibo`

```
fibo = [0, 1] ++ zipWith (+) fibo (drop 1 fibo)
```

After that, we append `zipWith (+) fibo (drop 1 fibo)`

This is self-referential: `fibo` refers to the list itself, and `drop 1 fibo` refers to the list itself but skipping the zeroth element.

`zipWith (+)` applies the `+` operation to the head of its two arguments, which start off at `0` and `1`.

This causes it to produce `1` for its first result.

But then it starts to consume itself. It produces 2, because `1 +` its first output is `2`. Because this function is lazy, it's able to pull from lists that it is creating, indefinitely.

Questions?

Knowledge check 3

1. Define an element-wise multiplication function using `zipWith`. That is `mul [1, 2, 3] [4, 5, 6] == [1, 10, 18]`
2. Now define a dot product that treats the vectors as lists, using the `mul` function you just defined. The dot product of two vectors `a` and `b` is `ax * bx + ay * by + az * bz + ...`. That is, we multiply each component together and we sum all the piecewise multiplications together.

KC 3 answers

1.

```
mul :: [Integer] -> [Integer] -> [Integer]
mul = zipWith (*)
```

2.

```
dot :: [Integer] -> [Integer] -> Integer
dot x y = sum $ x `mul` y
```

Questions?

What about the opposite?

If there is a zip, which takes two lists and makes a list of pairs, is there also an unzip?

[What do you think?]

Of course there is!

And if there weren't, we could make it ourselves.

`unzip [(1, 'a'), (2, 'b'), (3, 'c')]` should be `([1,2,3], ['a', 'b', 'c'])`

Right?

What would its type be?

`unzip`'s type

```
unzip :: [(a, b)] -> ([a], [b])
```

"Give me a list of pairs, I will give you a pair of lists."

How do we code it?

Start recursively...

`unzip`

```
unzip' :: [(a, b)] -> ([a], [b])
unzip' [] = ([], [])
unzip' ((x, y) : rest) =
    let (xs, ys) = unzip' rest
    in  (x : xs, y : ys)
```

The wrinkle here is that we need to bind the `xs` and `ys` from the recursive call so we can push new values onto them.

You might have preferred the recursive solutions to these built-in operators so far, but I think you'll like the one using fold for this one.

First, which fold do we use? `foldl` or `foldr`?

`unzip` (2)

`foldl` would end up reversing the order (trace through it to see why!). We use `r`.

```
unzip'' :: [(a, b)] -> ([a], [b])
unzip'' = foldr (\(x, y) (accx, accy) -> (x : accx, y : accy)) ([], [])
```

Here, we start with an empty pair of lists. The binary function appends each of the pair in the original list to the appropriate value in the accumulator.

When we're done, the accumulator holds our result.

It ends up being in the right order because we push from right to left. `foldl` would push from left to right and flip the order!

Is `unzip` the inverse of `zip`?

We say that `g` is the inverse of `f` if `g . f` is equivalent to `id`.

`id` is the identity function. `id x = x`. It's just the lambda that returns its argument without doing anything.

If applying `g` after `f` is the same as doing nothing at all, `g` is an inverse of `f`.

It seems like `unzip` "undoes" `zip`. Is that true?

Not quite

We can do this: `unzip $ zip xs ys` and we get back `(xs, ys)`

It's not quite the same though. We get the arguments packed into a pair, whereas previously they weren't stored that way.

The identity function has a definite meaning. It's a unary function. The issue is that `zip` is a binary function. So `unzip . zip` isn't really the same as `id`.

What about the other way around? Is `zip` the inverse of `unzip`?

No

`unzip` returns a tuple, but zip takes two singles, so `zip . unzip` is a type error. We can't compose those two functions.

There is a way to do it, though. `zip` normally takes 2 arguments.

Is there a way we can make it take one? Specifically, one pair of two lists?

Yes

Remember `curry` and `uncurry`? Here, we want to `uncurry zip`.

`uncurry zip . unzip` is equivalent to `id`.

That is, call unzip on a list of pairs, and then called the uncurried version of zip on the result to zip them back together.

Example:
`(uncurry zip . unzip) [(1,'a'), (2,'b'), (3, 'c')]` returns
`[(1,'a'), (2,'b'), (3, 'c')]`, the same thing that `id` returns.

Composability

Remember that `.` is the composition operatorer.

The reason we care about composition at all? Because in functional programming, it's the main way we build bigger functions out of smaller ones.

If two functions are compatible with each other (that is, we can pass the result of one to the other), we call them "composable".

At the very least, we need type-composability. That is, if `f` and `g` are functions, then:

```
f :: a -> b
g :: b -> c
f . g :: a -> c
```

Composability (2)

Composability becomes an issue when we start making our programs more complicated.

For example, suppose I have a random function like `f = div 100`.

This function divides 100 by integers. So `f 25 == 4` and `f 10 == 10`.

Suppose I want to compose this function with the function `show` to make it a string.

So in `g = show . f`, `g` is a function that takes a number, divides it into 100, and then converts the result into a string. It is compatible with the type `Integer -> String`.

But here's a question: what happens if we write `g 0`? [What string do we get?]

Composability (3)

We don't get a string, we get an error:

`"*** Exception: divide by zero`

(there's a double quote there but it's not a string, it's the text in the exception)

The issue is that `div n` is a *partial function*. That is, it's a function that isn't defined on all its arguments. In this case, `0`.

So we don't have anything to give to show. `div n` is not perfectly composable.

We could return an optional value. So if there is a quotient, we return it, but if there isn't we return a "nothing" kind of value. What is a good candidate for that?

Maybe

Consider this version of division:

```
divMaybe :: Integer -> Integer -> Maybe Integer
divMaybe _ 0 = Nothing
divMaybe x y = Just $ x `div` y
```

Now there's no immediate problem:

```
divMaybe 100 25 == Just 4
divMaybe 100 0 == Nothing
```

Suppose `f = divMaybe 100` like before. [What is the type of `f`?]

Composing maybe

```
f :: Integer -> Maybe Integer
```

Which means now `g = show . f` stil works, but now it prints "Just"...

What if we only want to print the value if it's there and "Error" if it's not?

[Class, write that for me]

Composing maybe (2)

```
g (Just x) = show x
g Nothing = "Error"
```

Now `g $ f 25` returns `"4"`

And `g $ f 0` returns `"Error"`

Which is correct, but it also made `g` less elegant. We had to handle both cases.

Previously we just wrote `g = show . f`.

Composability (4)

Composability is a major part of functional software design.

We like functions that are composable.

Does that mean we don't like `Maybe`? Because it makes it harder to compose?

No, it turns out there's an easier way. We can compose with `Maybe`, we just have to extend our idea of `map` so that it works with `Maybe` instead of only lists.

What would that look like?

`map` on lists

Remember that `map` takes a function and a list and it returns a new list in which the function is applied to every element of the original list.

So `map show [1,2,3] == ["1", "2", "3"]`
and `map show [] == []`

And what is a `Maybe` but a list of either 1 or 0 elements?

`fmap`

The concept of "thing that we can map over" is more general than just a list.

In fact, we can do it to `Maybe`s, too.

Instead of `map`, we call this function `fmap`:

```
g :: Integer -> Maybe String
g = (fmap show) . f
```

Now, we get `Nothing` if the argument is `0`, and `Just` a string if the argument.

...which isn't what we want. We want it to say `"Error"`, and we want the result to be a `String`, not a `Maybe String`.

But before we fix that, let's briefly consider: why is it called `fmap`? What is the `f`?

The `f` in `fmap`

The `f` stands for `Functor`.

A `Functor` is anything that can be mapped on. Lists are functors, but so are `Maybe`s.

In fact, almost every data structure is a `Functor`. We frequently want to do something to ever member of a data structure, so it makes sense to map over them.

`Functor` is a term from category theory, an advanced and abstract area of mathematics. Haskell takes many terms from category theory. Frankly, I don't know very much about category theory, so I will explain these things differently. You don't need to know category theory in order to make sense of Haskell. (Although it probably wouldn't hurt).

Fixing the function

Okay, so `g = (fmap show) . f`

But we don't want a `Maybe String`. We want a regular `String`, and if there isn't one, we want it to be replaced by `"Error"`

This is a function we could easily write. It replaces `Nothing` with a default value:

```
replaceNothing :: a -> Maybe a -> a
replaceNothing def Nothing = def
replaceNothing def (Just x) = x
```

This function already exists. It's called `fromMaybe`:

```
import Data.Maybe
fromMaybe :: a -> Maybe a -> a
```

Fixing the function (2)

So now we can finally fix it:

```
g :: Integer -> String
g = (fromMaybe "Error") . (fmap show) . f
```

If `f` returns `Just` a number, `fmap show` turns that into `Just` a string, and `fromMaybe` turns it into a regular string.

Is that good?

So it's more complicated now, but that makes sense: we added more requirements.

But it's not *dramatically* more complicated. It's still just a chain of compositions. There still aren't any `if` expressions in our final answer.

`divMaybe` has an implicit one, but it also kind of needs one: `0` is a special case.

Managing complexity in an object-oriented language is more about encapulating code into classes. But here, we don't really hide anything. We just put together a pipeline that does what we want.

Functional programming instruction focuses a lot on immutability and functions (obviously), but to me, the core of functional programming is composition and creating "pipelines" of code.

Questions?

Odds and ends

These are just some important odds and ends. You need to know these things, and they are in the required reading, but I didn't know where to put them in lecture...

Indexing lists (`!!`)

If you want to get, e.g., the 3rd element of a list (starting from 0), you can write `1 !! 3`.

So `[0, 1, 2, 3, 4, 5] !! 3` `==` `3`

Practice: treat the `(!!)` operator as a function and define it. What is the base case?

`words`

There is a function that lets you take a `String` and split it into a list of `String`s on whitespace. It's called `words`:

```
words :: String -> [String]
words "Hello, world! How are you today?" ==
    ["Hello,","world!","How","are","you","today?"]
```

What if we want to split on something other than whitespace?

Surprisingly, there isn't a built-in "split" method that lets us split on other things besides whitespace.

Practice: make one.

`unwords`

The opposite of `words`. Takes a list of strings and pastes them together with single spaces in between:

```
unwords ["hello", "world", "hi"] == "hello world hi"
```

`intercalate`

What if we want to paste them together with something other than a single space?

We can use the `intercalate` function:

```
import Data.List; -- needed for intercalate. It works with any list.

intercalate :: [a] -> [[a]] -> [a] -- get that?

--example:
intercalate ";;" ["hello", "world", "hi"] ==
    "hello;;world;;hi"
```

Practice: write this function!

`lines`

Splits a string into lines instead of words:

```
lines ::
lines "hello\nworld" == ["hello", "world"]
```

Practice: you know what to do!

Bonus points: make it work with both "\n" and "\r\n" (Windows line endings)

`unlines`

And of course there's an unlines:

```
unlines :: [String] -> String
unlines ["hello", "world"] == "hello\nworld\n"
```

Notice that extra "\n" at the end. `unlines` does that for some reason.

Practice: ...?

Take and drop

`take :: Int -> [a] -> [a]` takes n values from the list.

`drop :: Int -> [a] -> [a]` skips over the next n values from the list.

This is useful for working with infinite lists:

```
ghci> squares = map (^2) [1..]
ghci> take 10 squares
[1,4,9,16,25,36,49,64,81,100]
```

Suppose I want the first 10 squares after 100:

```
ghci> take 10 $ drop 10 squares
[121, 144, 169, 196, 225, 256, 289, 324, 361, 400]
```

Practice: I think we already wrote these, but do it again!

(take|drop)While

Recall that, in programming, a predicate is a function that returns a Bool. `takeWhile` takes a predicate and a list, and grabs values from it as long as the predicate is true.

```
ghci> takeWhile (isEven) [2,4,6,1,2,3]
[2,4,6]
```

But it stops as *soon* as the predicate fails:

```
ghci> takeWhile (isEven) [1,2,3,4,5,6]
[]
```

Here, it failed on the very first value, so it returned the empty list. That's how `takeWhile` is different from `filter`. This is a great one to write yourself as practice.

(take|drop)While (2)

There is also `dropWhile`, which drops as long as a predicate is true:

```
ghci> dropWhile (isEven) [2,4,6,1,2,3]
[1,2,3]
```

Now, as practice, try writing `takeUntil`, which takes as long as the predicate is *false*.

You can write this using function composition. Remember that `not` is a function.

Questions?

Required reading

More on datatypes

Other data structures (note: this is a tough one, but it's only reading. No exercises.)

Quiz format

You know the drill! 15 minutes, and 4 questions, each worth 25%. These questions will focus on functional operators:

1. `map` or `fmap`
2. `filter`
3. `zip`
4. `fold` and friends
5. Point-free style with the above
6. Currying and uncurrying
7. The odds-and-ends functions

Practice quiz 1

Include types for all of these functions. You can define intermediate functions that are not point-free, but your solution functions must be point free.

1. Define a function *point free* named *uppify* which converts a string to upper case, using the `toUpper` function. So `uppify "heLLo" == "HELLO"`. Your solution must be point free to receive credit.
2. Define a function `replacify` which uses `isLower` and replaces all the lowercase letters with a question mark. So `replacify "heLLo" == ??LL?`
3. Use `filter` define a point free function `countUppers`, which uses `isUpper` and `length` to count the number of upper case characters in a string. `countUppers "heLLo" == 2`. Your solution must be point free to receive credit.
4. Do the same thing as 3 using `foldl` and `map` and not using `filter`.

Practice quiz 1 answers

```haskell
-- 1.
uppify :: String -> String
uppify = map toUpper
-- 2.
replacify :: String -> String
replacify = map (\x -> if isLower x then '?' else x)
-- 3.
countUppers :: String -> Int
countUppers = length . filter isUpper
-- 4.
countUppers' :: String -> Int
countUppers' = foldl (+) 0 . map oneIfUpper
    where
        oneIfUpper x = if isUpper x then 1 else 0
```

Practice quiz 2

Include types for all of these functions. 1 and 4 must be point free for any credit.

1. Write a function **point-free** which returns the first letter of every word in a given string (assume non-empty). For example: `firstLetters "hello world!" == "hw"`.
2. Write a function (not point free) that takes a pair of strings, converts them to a list of words, and then counts the number of those word pairs that are identical. So `sameWordCount "hello world how is it?" "hello world is it good?" == 2` because "hello" and "world" are the same word in the same position in both. (note: the "it" is the 5th word in the first string and the 4th in the second: doesn't count)
3. Write a function that determines if two strings have the same length
4. Write a function **point free** that determines the longest line in a string with newlines in it. Function must be point free to receive credit. You can use `maximum :: [Int] -> Int` which returns the largest value of a list.

Practice quiz 2 answers

```haskell
-- 1.
firstLetters :: String -> String
firstLetters = map head . words
-- 2.
sameWordCount :: String -> String -> Int
sameWordCount s t = length $ filter (== True) $ zipWith (==) (words s) (words t)
-- 3.
sameLength :: String -> String -> Bool
sameLength s t = length s == length t
-- 4.
longestLineLength :: String -> Int
longestLineLength = maximum . map length . lines
```

Practice quiz 3

Include types for all these functions. If it says define a point-free function, it must be point-free for credit.

1. Suppose we had a function `isPrime :: Int -> Bool` that took an `Int` and returned whether or not it was prime. Define an infinite list of the prime numbers.
2. Define a function `sumOfFirstPrimes` that takes `n` and returns the sum of the first `n` primes (the number of primes is the argument). For example `sumOfFirstPrimes 3 == 10`, becuase the first 3 primes are 2, 3, and 5, and the sum 2 + 3 + 5 is 10.
3. Define a function `whichPrime` which, given a positive `Int` `n`, returns which prime number it is, starting from 1 (assume `n` is prime) So `whichPrime 5 == 3` (hint: try using `takeWhile` on the list of primes by those less than the given number)
4. Define a **point-free** function that returns whether or not an `Int` is even.

Practice quiz 3 answers

```haskell
-- 1.
primes :: [Int]
primes = filter isPrime [1..]
-- 2.
sumOfFirstPrimes :: Int -> Int
sumOfFirstPrimes n = sum $ take n primes
-- 3.
whichPrime :: Int -> Int
whichPrime n = length $ takeWhile (<=n) primes
-- 4.
isEven :: Int -> Int
isEven = (== 0) . (`mod` 2)
```

Practice quiz 4

Include types for all these functions. If it says define a point-free function, it must be point-free for credit.

1. Write a function `last` which returns the last element of any type of list if it exists, or `Nothing` if the list is empty. Do not return a list.
2. Use this function to define **point-free** a function named `endsWithQuestion`. You must handle `Maybe`s correctly.
3. Now define a function **point-free** named `any'` which returns true if *any* element of a list of bools is true, and false otherwise (this function already exists, which is why there's a `'`)
4. Now define a function **point-free** called anyLineEndsWithQuestion, using `any'`, `endsWithQuestion`, and any other useful functions you know or define.

Practice quiz 4 answers

```haskell
import Data.Maybe -- <- you wouldn't need to write this on a real quiz
-- 1.
last :: [a] -> Maybe a
last l = case drop (length l - 1) of
        (h : _) -> Just h
        _ -> Nothing
-- 2.
endsWithQuestion :: String -> Bool
endsWithQuestion = (== '?') . fromMaybe '_' . last
-- 3.
any' :: [Bool] -> Bool
any' = not . null . filter (== True)
-- I made a small mistake on this one originally, using '$' instead of '.'.
-- That would require the right argument of not $ ... to be a boolean
-- -5 points from me!
-- 4.
anyLineEndsWithQuestion :: String -> Bool
anyLineEndsWithQuestion = any' . map endsWithQuestion . lines
```

Ask an AI

Ask an AI to generate a practice quiz for you like the above! Give it the slides starting with "# Quiz Format" and up to and including this slide. Or give it the whole markdown.

Then, ask it to grade you. I like to use this scale:

1. 0 points off for extremely minor things. Misspellings or missing grouping operators that are clearly intended.
2. 5 points for mistakes that cause the code to fail but are more than just minor mistakes. For example a small type error where it's clear you get the big idea but, e.g., applied the applicative to too many arguments or something.
3. 10 points for bigger mistakes, like type errors that can't work, but there's still "more than half" of the understanding demonstrated.
4. Zero points total if there are several major mistakes or it looks like you're guessing.

Questions?